



Module 41-46: Engineering Math for AI - Complete Notes



What You'll Learn

Master **sequences, functions, vectors, matrices, and counting** — the math behind AI and advanced testing.



Module 41-42: Sequences & Functions

Sequences

```
# Arithmetic Sequence: Add same amount each time
# 2, 5, 8, 11, 14... (add 3)

def arithmetic_sequence(start, step, n):
    """Generate n terms of arithmetic sequence"""
    return [start + i * step for i in range(n)]

print(arithmetic_sequence(2, 3, 5)) # [2, 5, 8, 11, 14]

# Geometric Sequence: Multiply by same amount
# 2, 6, 18, 54... (multiply by 3)

def geometric_sequence(start, ratio, n):
    """Generate n terms of geometric sequence"""
    return [start * (ratio ** i) for i in range(n)]

print(geometric_sequence(2, 3, 5)) # [2, 6, 18, 54, 162]
```

Linear Functions

```
# y = mx + b (slope-intercept form)
# m = slope (rate of change)
# b = y-intercept (starting point)

def linear_function(x, m, b):
    """Calculate y = mx + b"""
    return m * x + b

# Cost = $5 per item + $20 fixed
cost = linear_function(10, 5, 20) # 70 dollars for 10 items
```



Module 43-44: Vectors & Matrices

Vectors

```
# Vector: List of numbers representing a point/direction

def vector_add(v1, v2):
    """Add two vectors"""
    return [a + b for a, b in zip(v1, v2)]

def vector_magnitude(v):
    """Length of vector"""
    return sum(x**2 for x in v) ** 0.5

def dot_product(v1, v2):
    """Dot product of vectors"""
    return sum(a * b for a, b in zip(v1, v2))

v1 = [1, 2, 3]
v2 = [4, 5, 6]
print(vector_add(v1, v2))      # [5, 7, 9]
print(vector_magnitude(v1))   # 3.74
print(dot_product(v1, v2))    # 32
```

Matrices

```
# Matrix: 2D array of numbers

def create_matrix(rows, cols, default=0):
    """Create matrix filled with default value"""
    return [[default] * cols for _ in range(rows)]

def matrix_dimensions(m):
    """Get rows x cols dimensions"""
    return (len(m), len(m[0]) if m else 0)

def matrix_multiply(a, b):
    """Simple matrix multiplication"""
    rows_a, cols_a = matrix_dimensions(a)
    rows_b, cols_b = matrix_dimensions(b)

    if cols_a != rows_b:
        return None # Can't multiply

    result = create_matrix(rows_a, cols_b)
    for i in range(rows_a):
        for j in range(cols_b):
            for k in range(cols_a):
                result[i][j] += a[i][k] * b[k][j]
    return result

A = [[1, 2], [3, 4]]
```

```
B = [[5, 6], [7, 8]]
print(matrix_multiply(A, B)) # [[19, 22], [43, 50]]
```

Module 45: Distance Metrics (for AI)

```
# Cosine Similarity: How similar are two vectors?
# Used in text similarity, recommendations

import math

def cosine_similarity(v1, v2):
    """Similarity between 0 (different) and 1 (identical)"""
    dot = dot_product(v1, v2)
    mag1 = vector_magnitude(v1)
    mag2 = vector_magnitude(v2)

    if mag1 == 0 or mag2 == 0:
        return 0
    return dot / (mag1 * mag2)

# Similar direction = high score
v1 = [1, 0, 1]
v2 = [1, 0, 1]
print(cosine_similarity(v1, v2)) # 1.0 (identical)

v3 = [0, 1, 0]
print(cosine_similarity(v1, v3)) # 0.0 (perpendicular)
```

Module 46: Counting Techniques

```
# Permutations: Order matters
# How many ways to arrange 3 items from 5?

import math

def permutations(n, r):
    """P(n,r) = n! / (n-r)!""""
    return math.factorial(n) // math.factorial(n - r)

print(permutations(5, 3)) # 60

# Combinations: Order doesn't matter
# How many ways to choose 3 items from 5?

def combinations(n, r):
    """C(n,r) = n! / (r!(n-r)!""""
    return math.factorial(n) // (math.factorial(r) * math.factorial(n - r))
```

```

print(combinations(5, 3)) # 10

# Practical: Test Case Combinations
def test_combinations(options_per_field):
    """Total test case combinations for multiple fields"""
    total = 1
    for options in options_per_field:
        total *= options
    return total

# 3 browsers × 4 OS × 2 screen sizes
print(test_combinations([3, 4, 2])) # 24 combinations

```

SDET/Testing Application

```

# SDET Scenario: Embedding Similarity for Test Matching

def find_most_similar(query_vector, document_vectors):
    """Find most similar document to query"""
    similarities = []
    for i, doc in enumerate(document_vectors):
        sim = cosine_similarity(query_vector, doc)
        similarities.append((i, sim))

    similarities.sort(key=lambda x: x[1], reverse=True)
    return similarities

# Error message embeddings (simplified)
error_vectors = [
    [1, 0, 1, 0], # Error 0: login related
    [0, 1, 0, 1], # Error 1: database related
    [1, 1, 0, 0], # Error 2: auth related
]

new_error = [1, 0.5, 0.5, 0] # New error
similar = find_most_similar(new_error, error_vectors)
print(f"Most similar to error {similar[0][0]}")

```

```

# SDET Scenario: Pairwise Testing

from itertools import combinations

def generate_pairwise(options_dict):
    """Generate pairwise test combinations"""
    keys = list(options_dict.keys())
    all_pairs = []

```

```
# Get all pairs of parameters
for param1, param2 in combinations(keys, 2):
    for val1 in options_dict[param1]:
        for val2 in options_dict[param2]:
            all_pairs.append({param1: val1, param2: val2})

return all_pairs

options = {
    "browser": ["chrome", "firefox"],
    "os": ["windows", "mac"],
    "theme": ["light", "dark"]
}

pairs = generate_pairwise(options)
print(f"Pairwise combinations: {len(pairs)}") # 12 instead of 8
```

Key Takeaways

- ✓ **Arithmetic sequence:** Add constant
- ✓ **Geometric sequence:** Multiply constant
- ✓ **Linear function:** $y = mx + b$
- ✓ **Vectors:** Lists for points/embeddings
- ✓ **Cosine similarity:** Compare vectors (0-1)
- ✓ **Permutations:** Order matters
- ✓ **Combinations:** Order doesn't matter

 Save as: `Module_41_46_Engineering_Math.md`

Phase 8 Complete! 